

IoT-Enabled Energy Monitoring and Load Protection System

Technical Project Report

ESP32 • PZEM-004T • Blynk IoT • SIM900A • SW-420 • Relay • EEPROM

Contents

1. Abstract
2. Introduction
3. Proposed System
4. System Architecture
5. System Flow
6. Hardware Design
7. Circuit and Pin Configuration
8. Firmware and Code Logic
9. Output
10. Prototype
11. Conclusion

1. Abstract

The IoT-Enabled Energy Monitoring and Load Protection System is an ESP32-based embedded system designed to monitor electrical parameters, provide local and remote visibility, and automatically control an electrical load when configured operating conditions are exceeded. A PZEM-004T module supplies electrical measurements including voltage, current, power, energy, frequency, and power factor to the ESP32 through UART. The measured information is presented on a 16×2 I2C LCD and transmitted to a Blynk IoT dashboard through the ESP32's Wi-Fi connectivity. A relay provides load control, while an SW-420 vibration sensor provides physical tamper detection. The SIM900A module provides GSM-based SMS communication for system alerts and interaction. EEPROM-backed storage is used to retain important operating data across power interruptions. The firmware coordinates these functions using periodic, timer-based tasks within the ESP32 main loop.

2. Introduction

Electrical loads require continuous monitoring when the objective is to understand consumption and prevent operation beyond a configured power limit. Conventional local measurement provides only limited visibility, while a connected embedded system can combine measurement, local display, remote monitoring, alert communication, and automatic load control in a single platform.

This system uses the ESP32 as the central controller. The PZEM-004T is used for electrical measurement, the SW-420 is used for vibration-based tamper detection, and a relay provides the load-control interface. The system also integrates a 16×2 I2C LCD, SIM900A GSM communication, Wi-Fi connectivity, Blynk IoT, and EEPROM-based persistence. The resulting implementation provides a compact monitoring and protection platform with both local and remote interfaces.

3. Proposed System

The proposed system is organized around an ESP32 microcontroller that receives inputs, processes operating conditions, updates the load-control output, and distributes system information to local and remote interfaces.

- Electrical measurement: PZEM-004T provides voltage, current, power, cumulative energy, frequency, and power factor.
- Local indication: a 16×2 I2C LCD presents electrical values, load status, and priority system alerts.
- Load protection: the relay is controlled according to the configured power limit and system operating state.
- Tamper detection: the SW-420 vibration sensor detects physical movement or vibration and initiates a GSM alert.
- GSM communication: SIM900A is used for SMS-based alerts and system interaction.
- IoT monitoring: ESP32 Wi-Fi sends electrical and system values to the Blynk IoT dashboard.
- Data persistence: important operating values are periodically stored in the ESP32 EEPROM emulation.

4. System Architecture

The architecture is divided into four functional groups: inputs and sensors, the ESP32 controller, outputs and control, and communication/IoT interfaces. The ESP32 is the central processing element. The PZEM-004T communicates through UART, while the SW-420 provides a digital input. The relay and LCD are controlled by the ESP32, and the SIM900A provides a separate UART communication path. Wi-Fi connects the controller to Blynk IoT for remote monitoring.

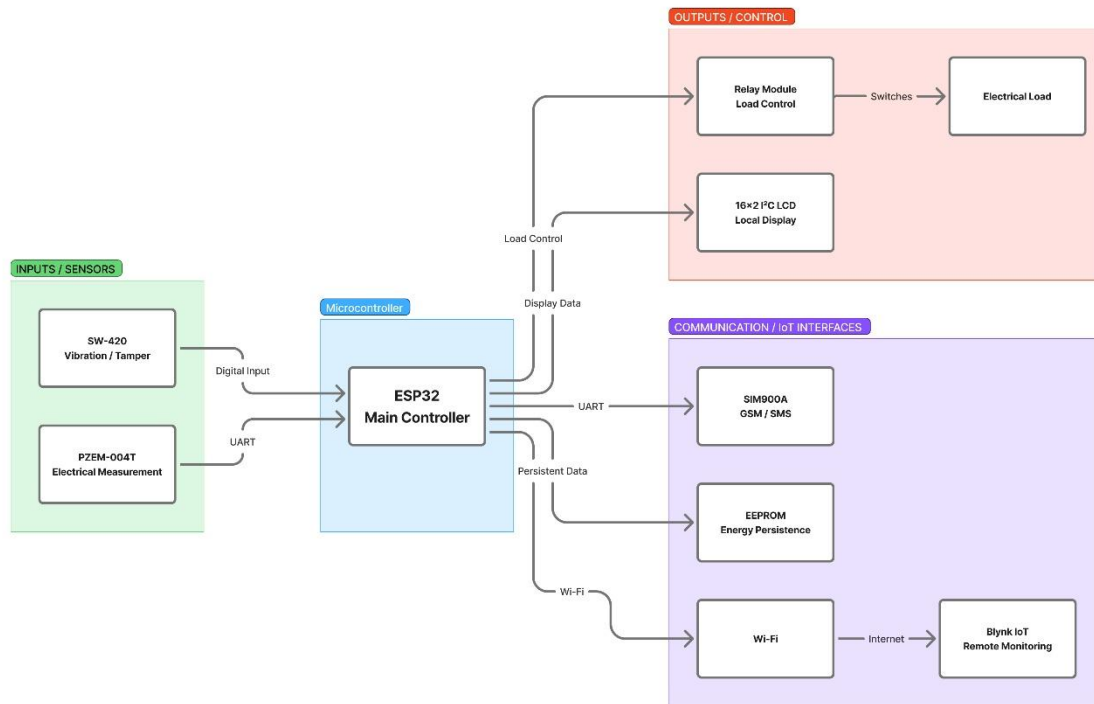


Figure 1. Overall system architecture.

5. System Flow

At startup, the ESP32 initializes its GPIO interfaces, EEPROM, LCD, GSM module, Wi-Fi connection, and Blynk communication. During normal operation, electrical data is read periodically from the PZEM-004T. The controller evaluates the measured power against the configured limit and updates the relay state. The vibration input is continuously monitored for tamper events. Local display information and Blynk data are updated at their respective intervals, while EEPROM storage is refreshed periodically.

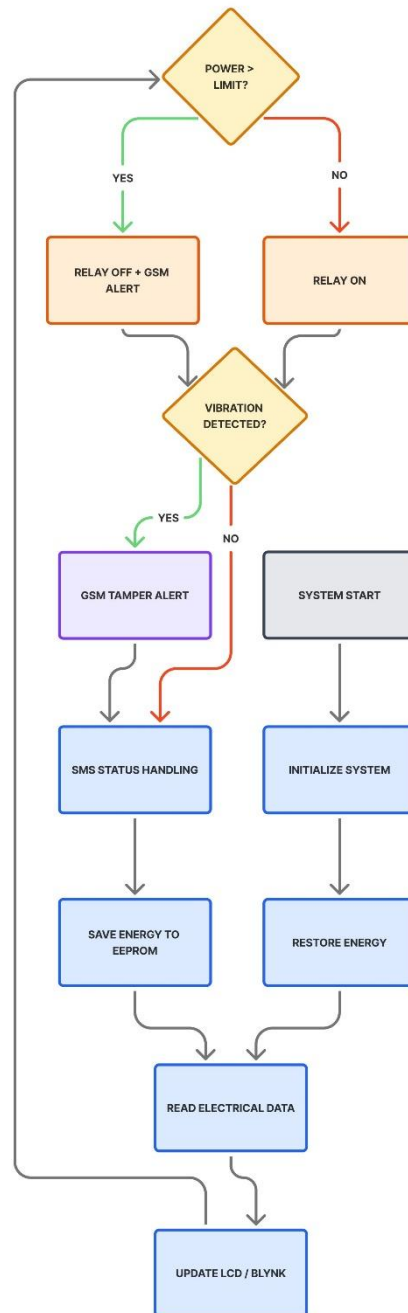


Figure 2. Overall system flowchart.

6. Hardware Design

The hardware is centered on an ESP32 DevKit V1. Measurement and sensing devices are placed on the input side of the controller, while the relay and LCD form the local output/control interface. The SIM900A provides GSM communication, and the ESP32's integrated Wi-Fi interface provides the IoT connection.

Component	Role	Interface
ESP32 DevKit V1	Main controller and processing unit	GPIO / UART / I2C / Wi-Fi
PZEM-004T v3.0	Voltage, current, power, energy, frequency and PF measurement	UART
SW-420	Vibration / tamper detection	Digital input
Single-channel relay	Electrical load switching and protection control	GPIO
16×2 I2C LCD	Local display of system parameters and status	I2C
SIM900A	GSM SMS alerts and system interaction	UART
EEPROM emulation	Persistent storage of important operating data	ESP32 internal flash
Blynk IoT	Remote visualization of system data	Wi-Fi / Internet

6.1 Power Arrangement

The circuit uses a 12 V DC input supply. The illustrated power arrangement uses an LM2596 buck converter to provide a 5 V rail and a 3.3 V regulator for the low-voltage sensor interface. A common ground reference is used between the controller and connected modules.

7. Circuit and Pin Configuration

The circuit connects each peripheral directly to the ESP32 using dedicated GPIO assignments. The PZEM-004T and SIM900A use separate hardware UART interfaces, the LCD uses the ESP32 I2C bus, and the relay and vibration sensor use dedicated digital GPIOs.

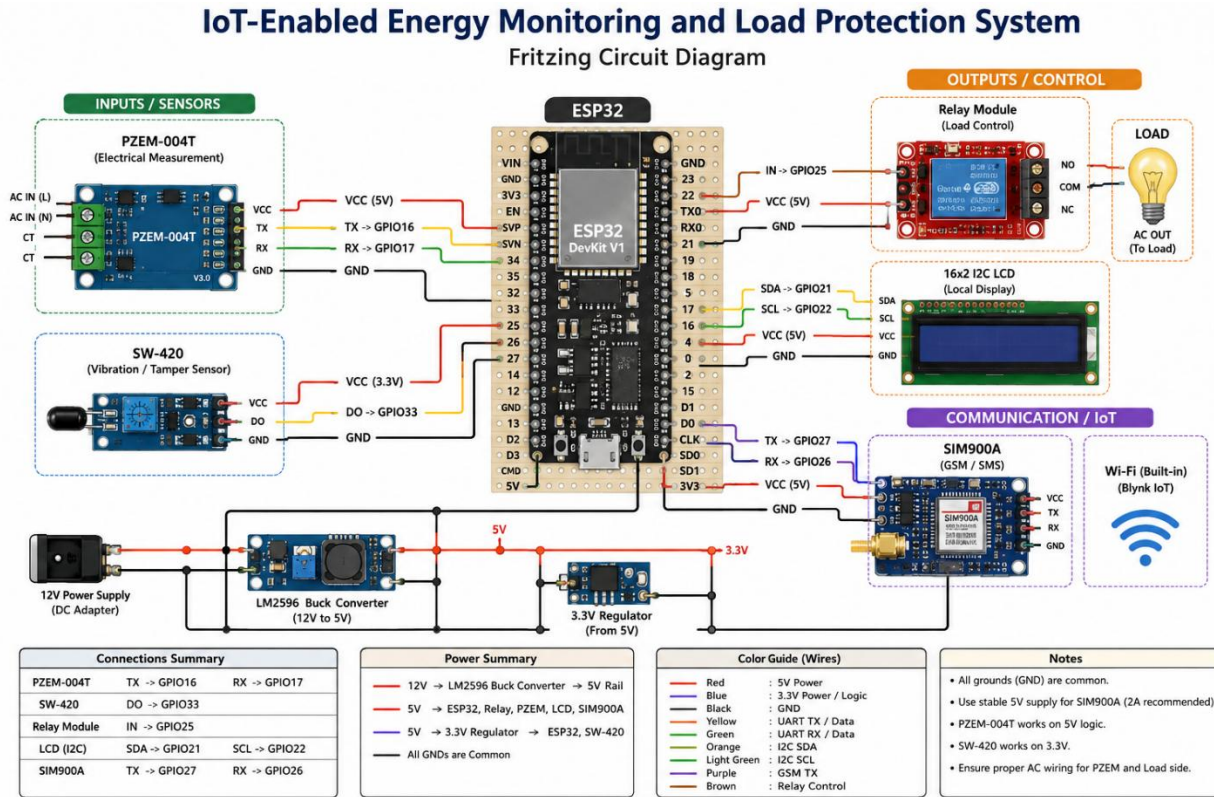


Figure 3. Overall circuit and wiring diagram.

Device	Signal	ESP32 GPIO	Interface / Function
PZEM-004T	TX	GPIO 16	ESP32 RX / UART2
PZEM-004T	RX	GPIO 17	ESP32 TX / UART2
SIM900A	TX	GPIO 27	ESP32 RX / UART1
SIM900A	RX	GPIO 26	ESP32 TX / UART1
Relay	Control	GPIO 25	Digital output
SW-420	DO	GPIO 33	Digital input
16x2 I2C LCD	SDA	GPIO 21	I2C data
16x2 I2C LCD	SCL	GPIO 22	I2C clock

8. Firmware and Code Logic

The firmware is written for the ESP32 using the Arduino framework. The implementation is organized into dedicated functions for measurement, load control, display handling, GSM communication, Blynk updates, EEPROM storage, Wi-Fi recovery, and tamper detection. Periodic operations are scheduled using millisecond timers so that the main loop can service multiple subsystems.

8.1 Initialization

- Initializes the serial monitor and GPIO modes.
- Initializes EEPROM and restores stored operating data.
- Initializes the 16×2 I2C LCD and custom display characters.
- Initializes the SIM900A UART and configures SMS operation.
- Connects the ESP32 to Wi-Fi and configures Blynk communication.

8.2 Electrical Measurement

The `readEnergy()` function obtains voltage, current, power, energy, frequency, and power factor from the PZEM-004T. Measurements are requested once every second. Invalid voltage, current, or power readings are treated as communication failures and the previous valid values are retained.

8.3 Load Protection

The `controlRelay()` function compares measured power with the configured `watt_limit`. When the measured power exceeds the limit, an overload condition is raised, an SMS alert is generated once for the event, and the relay is switched to disconnect the load. When the overload condition clears, normal relay control can resume. The relay control output is assigned to GPIO 25.

8.4 Tamper Detection

The SW-420 sensor is read through GPIO 33. A vibration event activates the tamper state and generates an SMS notification. A debounce interval is used to prevent repeated triggering from a single physical event.

8.5 Local Display

The LCD is updated every two seconds and rotates through three information pages. The pages display voltage/current/frequency/power factor, power and system status values, and cumulative energy with relay status. Priority messages such as overload and tamper conditions take precedence over normal display pages.

8.6 GSM Communication

The SIM900A is controlled through UART1. The firmware initializes the modem in text SMS mode and uses AT commands for message handling. System conditions such as overload and tamper events generate SMS notifications to the configured recipient. The firmware also supports SMS-based system interaction.

8.7 Blynk IoT Communication

When Wi-Fi is available, the firmware sends the current electrical and system values to Blynk every two seconds. The configured virtual channels are used for voltage, current, power, energy, system state, frequency, power factor, and relay state.

Virtual Pin	Data
V0	Voltage
V1	Current
V2	Power
V3	Energy
V4	System operating value
V5	Frequency
V6	Power factor
V7	Relay state

8.8 EEPROM Persistence

The ESP32 EEPROM emulation is initialized with a validity marker and used to retain important operating data. Stored values are written periodically and also during relevant state changes, allowing the system to restore retained information after a restart or power interruption.

9. Output

The Blynk IoT dashboard provides a remote view of the electrical parameters and relay state. The displayed interface includes voltage, current, real power, and energy values, together with individual relay controls/status indicators. The dashboard therefore provides a compact remote interface for observing the system.



Figure 4. Blynk IoT monitoring interface.

10. Prototype

The prototype integrates the ESP32 controller, measurement and communication modules, relay interface, display, power circuitry, and connected electrical load on a physical assembly. The photographs show the implemented hardware arrangement and wiring used for the system.

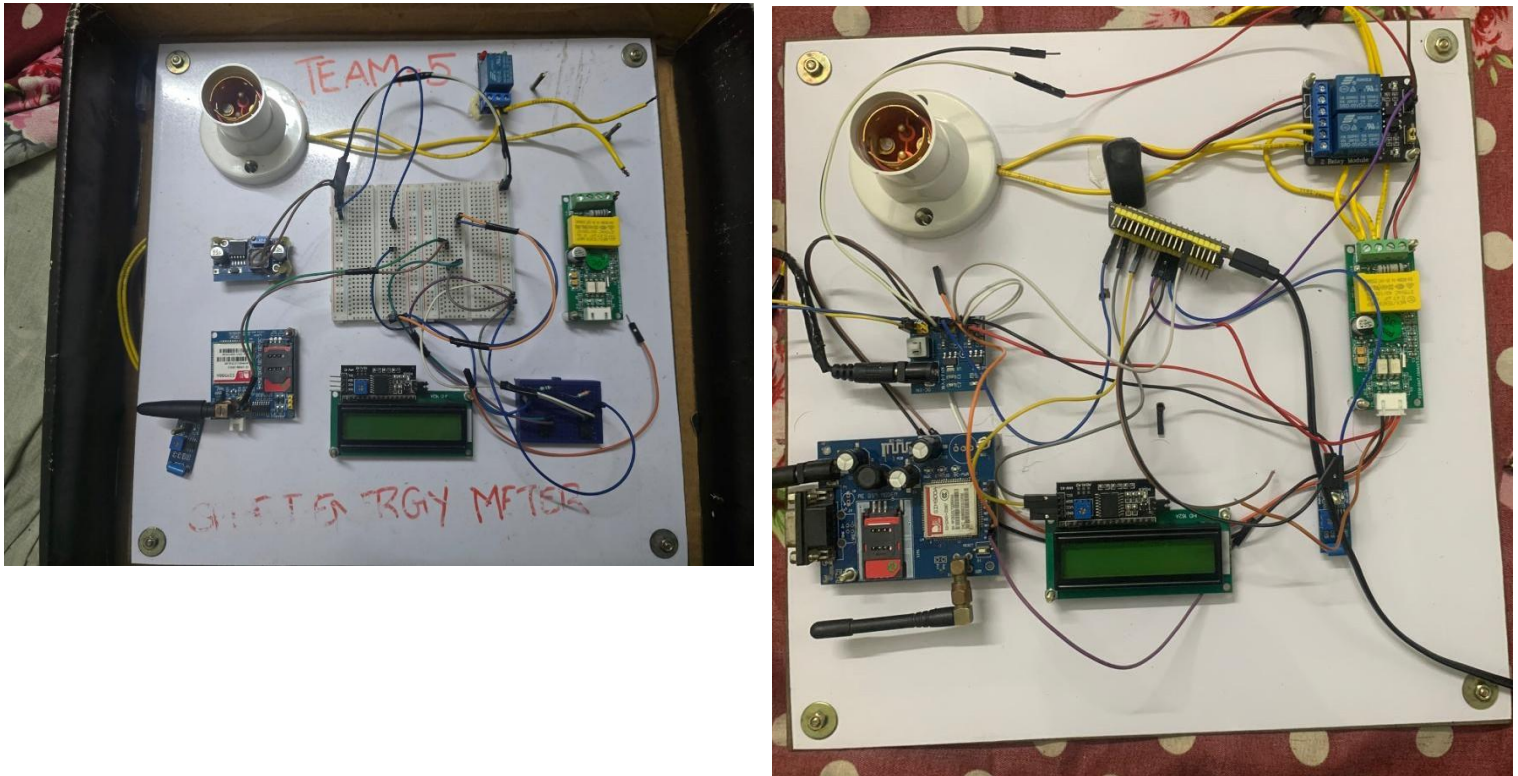


Figure 5. Prototype hardware assemblies.

11. Conclusion

The implemented system combines electrical measurement, load protection, tamper detection, local display, GSM notification, IoT monitoring, and persistent data storage around an ESP32 controller. The firmware provides periodic acquisition of electrical parameters from the PZEM-004T, evaluates load conditions, controls the relay, monitors the vibration sensor, updates the LCD and Blynk dashboard, and maintains communication and stored operating data. The resulting prototype provides a compact connected platform for electrical energy monitoring and automated load protection.